DOCUMENTACION TECNICA

Sistema de Gestion de Stock

Service Integral

Proyecto:	Sistema de Gestion de Stock - Service Integral
Stack Backend:	Node.js + Express + MySQL 8.0
Stack Frontend:	HTML5 + CSS3 + JavaScript Vanilla
Storage:	Cloudinary (CDN de imagenes)
Tunneling:	Cloudflare Tunnel (cloudflared.exe)

Introduccion

El presente documento describe la arquitectura de software, el esquema relacional de datos, los endpoints de la API REST y las politicas de seguridad implementadas en el sistema de gestion de stock del taller Service Integral. El sistema permite administrar el inventario de repuestos, registrar usuarios tecnicos y administrativos, gestionar pedidos a proveedores y almacenar evidencias fotograficas de los productos en la nube.

Service Integral es una aplicacion web desacoplada de tipo cliente-servidor, desplegada localmente y expuesta a traves de un tunel seguro de Cloudflare, lo cual evita la apertura de puertos en el router y mantiene la integridad de la red privada del taller.

1. Arquitectura General del Sistema

El sistema implementa una arquitectura desacoplada basada en el patron cliente-servidor con cuatro componentes principales que se comunican entre si mediante protocolos bien definidos.

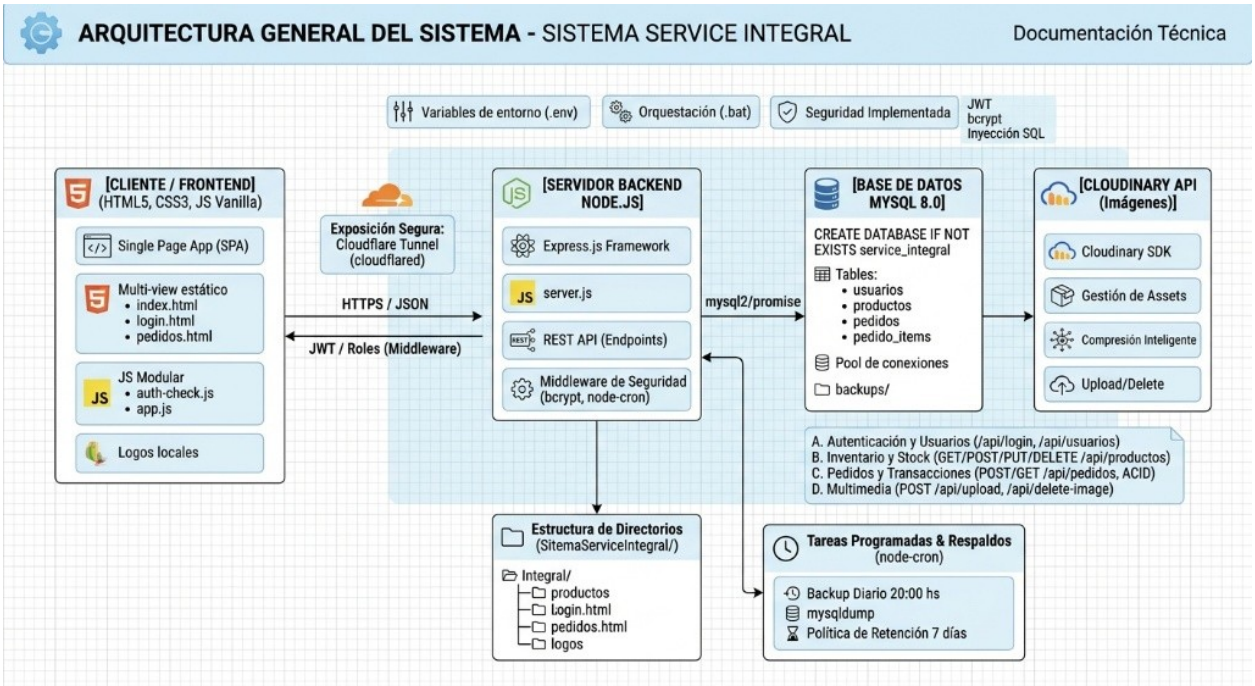


Figura 1. Diagrama de arquitectura general del Sistema Service Integral.

Componentes principales:

- Frontend: Single Page / Multi-view estatico desarrollado con HTML5 semantico, CSS3 estructurado en variables (style.css) y JavaScript Vanilla modular (auth-check.js, app.js).
- Backend: API REST desarrollada sobre el entorno de ejecucion Node.js utilizando el framework Express.js.
- Persistencia: Base de datos relacional MySQL 8.0 con pool de conexiones concurrentes (mysql2/promise).

- Storage Multimedia: Gestion de assets y compresion inteligente en la nube mediante Cloudinary SDK.
- Acceso Seguro / Tunneling: Exposicion segura de endpoints via Cloudflare Tunnel (cloudflared) sin requerir apertura de puertos en el router local.

2. Estructura de Directorios

A continuacion se detalla el arbol de directorios del proyecto, mostrando la separacion entre el backend, el frontend y los binarios de orquestacion del sistema.

Descripcion	Descripcion
`SitemaServiceIntegral/`	Carpeta raiz del proyecto
`backend/`	Logica de servidor y API REST
`.env`	Variables de entorno confidenciales
`package.json`	Dependencias del proyecto Node.js
`server.js`	Punto de entrada principal y API REST
`backups/`	Directorio de volcados .sql automaticos
`frontend/`	Interfaz de usuario (SPA estatica)
`index.html`	Tablero principal de Stock e Inventario
`login.html`	Modulo de autenticacion
`pedidos.html`	Gestion de ordenes de compra y pre-pedidos
`style.css`	Estilos globales y temas
`auth-check.js`	Middleware del frontend (JWT y Roles)
`logos/`	Recursos graficos locales
`cloudflared.exe`	Binario del tunel seguro Cloudflare
`Iniciar Sistema.bat`	Script de orquestacion y arranque en Windows

3. Modelo Relacional de Base de Datos (MySQL)

El esquema relacional se compone de cuatro tablas principales: usuarios, productos, pedidos y pedido_items. Las relaciones se mantienen mediante claves foraneas con reglas de integridad referencial.

```
CREATE DATABASE IF NOT EXISTS service_integral;
USE service_integral;

-- Tabla: usuarios
```

```
CREATE TABLE usuarios (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    rol ENUM('ADMIN', 'TECNICO') DEFAULT 'TECNICO',
    creado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Tabla: productos
CREATE TABLE productos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(150) NOT NULL,
    precio DECIMAL(10, 2) DEFAULT 0.00,
    stock INT DEFAULT 0,
    stockMinimo INT DEFAULT 0,
    categoria VARCHAR(50) DEFAULT 'Otros',
    distribuidor VARCHAR(100) DEFAULT NULL,
    codigoDistribuidor VARCHAR(100) DEFAULT NULL,
    descripcion TEXT DEFAULT NULL,
    imagen_1 VARCHAR(255) DEFAULT NULL,
    imagen_2 VARCHAR(255) DEFAULT NULL,
    imagen_3 VARCHAR(255) DEFAULT NULL,
    creado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Tabla: pedidos
CREATE TABLE pedidos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    notas TEXT DEFAULT NULL
);

-- Tabla: pedido_items
CREATE TABLE pedido_items (
    id INT AUTO_INCREMENT PRIMARY KEY,
    pedido_id INT NOT NULL,
    producto_id INT DEFAULT NULL,
    cantidad INT NOT NULL DEFAULT 1,
    FOREIGN KEY (pedido_id) REFERENCES pedidos(id) ON DELETE CASCADE,
    FOREIGN KEY (producto_id) REFERENCES productos(id) ON DELETE SET NULL
);
```

3.1 Notas sobre el esquema

- La columna rol de usuarios admite solo dos valores discretos: ADMIN o TECNICO. Por defecto se asigna TECNICO al crear un usuario nuevo.
- Las contraseñas se almacenan como hashes bcrypt (columna password con tamaño 255 caracteres).
- Un pedido se elimina en cascada (ON DELETE CASCADE): al borrar la cabecera, sus items asociados desaparecen automaticamente.
- Si se elimina un producto que figura en pedido_items, la referencia se establece en NULL (ON DELETE SET NULL) preservando el historial del pedido.
- Cada producto admite hasta tres URLs de imagenes (imagen_1, imagen_2, imagen_3) alojadas en Cloudinary.

4. Especificacion de la API REST (server.js)

La API sigue los principios RESTful y responde con payloads JSON. Todos los endpoints autenticados requieren un token Bearer en la cabecera Authorization.

4.A Autenticacion y Usuarios

POST /api/login

Payload de entrada:

```
{
    "email": "admin@serviceintegral.com",
    "password": "..."
}
```

Respuesta del servidor:

Objeto con JWT Token firmado (expiresIn: '8h') e informacion publica del usuario (id, nombre, rol).

POST /api/usuarios

Payload de entrada:

```
{
    "nombre": "...",
    "email": "...",
    "password": "...",
    "rol": "ADMIN" | "TECNICO"
}
```

Comportamiento: encripta la clave mediante bcrypt.hashSync(password, 10) e inserta el nuevo registro en la tabla usuarios.

4.B Inventario y Stock (CRUD)

GET /api/productos

Retorna el listado completo de repuestos ordenados de forma descendente por id (los mas recientes primero).

POST /api/productos

Inserta un nuevo repuesto con soporte para hasta 3 URLs de imagenes alojadas en Cloudinary.

PUT /api/productos/:id

Actualiza los atributos del producto especificado por parametro de ruta (id).

POST /api/productos/:id/stock-minimo

Actualiza de forma atomica el umbral de stock minimo del producto.

DELETE /api/productos/:id

Elimina fisicamente el repuesto de la base de datos (operacion destructiva, restringida a usuarios ADMIN).

4.C Pedidos y Transacciones

POST /api/pedidos

Utiliza transacciones ACID (connection.beginTransaction(), commit(), rollback()) para garantizar que la cabecera del pedido y el desglose de items se inserten de manera consistente. Al finalizar con exito, ejecuta un DELETE automatico de pedidos con antiguedad mayor a 2 meses.

GET /api/pedidos

Consulta el historial cronologico de ordenes de compra.

GET /api/pedidos/:id/items

Realiza un LEFT JOIN entre pedido_items y productos para retornar el detalle completo del pedido.

4.D Multimedia (Cloudinary)

POST /api/upload

Recibe el archivo binario en memoria RAM (multer.memoryStorage()), lo convierte a Base64 y lo envia a Cloudinary aplicando redimensionamiento preventivo (800x800), compresion automatica (quality: "auto") y conversion de formato (fetch_format: "auto").

POST /api/delete-image

Extrae el Public ID de la URL provista y destruye el archivo en el storage remoto (cloudinary.uploader.destroy).

5. Tareas Programadas y Politicas de Respaldo

El sistema implementa tareas en segundo plano mediante la libreria node-cron, responsables de automatizar los respaldos periodicos de la base de datos.

```
cron.schedule('0 20 * * *', () => {
    realizarBackup();
}, {
    scheduled: true,
    timezone: "America/Argentina/Buenos_Aires"
});
```

Detalle del job programado:

- Horario: diariamente a las 20:00 hs (hora de cierre comercial).
- Mecanismo: ejecuta un proceso hijo mediante child_process.exec invocando el binario mysqldump.exe para exportar la base completa a un archivo .sql con marca de tiempo.
- Politica de retencion: la rutina limpiarBackupsViejos elimina automaticamente los archivos de volcado con una antiguedad superior a 7 dias para evitar el crecimiento desmedido del almacenamiento.

6. Seguridad Implementada

Proteccion de Credenciales

Uso de bcryptjs con 10 rondas de salting para almacenar contraseñas hasheadas no reversibles. Esto garantiza que, aun accediendo a la base de datos, las claves originales no pueden recuperarse.

Tokens de Sesion (JWT)

Emision de JSON Web Tokens (JWT) firmados con clave secreta (JWT_SECRET) y caducidad temporal estricta. Los tokens expiran a las 8 horas, forzando al usuario a autenticarse nuevamente.

Prevencion de Inyeccion SQL

Todas las consultas a MySQL utilizan sentencias preparadas con placeholders parametrizados (?), lo cual impide que datos maliciosos inyectados en los inputs sean interpretados como SQL ejecutable.

Validacion Frontend y Control de Roles

Control de renderizado condicional en auth-check.js para evitar que usuarios no administradores accedan a funciones de mutacion o borrado de datos. Los botones sensibles se ocultan o desactivan segun el rol del token decodificado.

Fin del Documento

Sistema de Gestion de Stock - Service Integral | Documentacion Tecnica